

INSTITUTO TECNOLÓGICO SACABA

R.M. 995/2015

Fundado el 19 de marzo de 2007

Sacaba – Cochabamba - Bolivia

CARRERA DE SISTEMAS INFORMÁTICOS



“DESARROLLO DE UNA APLICACIÓN MÓVIL INTELIGENTE DE APOYO A LA TOMA DE DECISIONES PARA LA RECOMENDACIÓN DE MONTURAS Y CARACTERÍSTICAS DE LENTE ÓPTICOS EN LA ÓPTICA OCULIST”

Proyecto de Grado presentado para optar el Título de
Técnico Superior en Sistemas Informáticos

Postulante: Yeison Ramos Rioja

Tutor: Ing. Freddy Ledezma Higuera

Noviembre 2026
Sacaba, Cochabamba, Bolivia

INSTITUTO TECNOLÓGICO SACABA - ITSa

R.M. 995/2015

Fundado el 19 de marzo de 2007

Sacaba – Cochabamba - Bolivia

CARRERA SISTEMAS INFORMÁTICOS

R.M. Nº 0823/2019



“DESARROLLO DE UNA APLICACIÓN MÓVIL INTELIGENTE DE APOYO A LA TOMA DE DECISIONES PARA LA RECOMENDACIÓN DE MONTURAS Y CARACTERÍSTICAS DE LENTE ÓPTICOS EN LA ÓPTICA OCULIST”

Proyecto de Grado presentado para optar el Título de
Técnico Superior en Sistemas Informáticos

Postulante: Yeison Ramos Rioja

Tutor: Ing. Freddy Ledezma Higuera

Noviembre, 2026
Sacaba, Cochabamba, Bolivia

DEDICATÓRIA

A mi padre, por su apoyo incondicional, comprensión y cariño a lo largo de este camino y durante cada etapa de mi vida.

Su esfuerzo, sus consejos y su constante motivación fueron la fuerza que me impulsó a continuar, superar las dificultades y concluir este proyecto con dedicación, responsabilidad y esmero.

Este logro también le pertenece, porque su presencia y confianza en mí fueron fundamentales para alcanzar esta importante meta.

AGRADECIMIENTOS

Expreso mi más sincero agradecimiento al Instituto Tecnológico Superior de Sacaba y a la Carrera de Sistemas Informáticos, por brindarme la formación académica, los conocimientos y las herramientas necesarias para el desarrollo y culminación de este proyecto.

A mi docente tutor, Lic. Freddy Ledezma Higuera, por su orientación, paciencia, compromiso y valioso acompañamiento durante cada una de las etapas del trabajo, compartiendo sus conocimientos y recomendaciones para mejorar el desarrollo de esta investigación.

A todos los docentes de la Carrera de Sistemas Informáticos, por su dedicación, enseñanza y esfuerzo constante, que contribuyeron significativamente a mi formación académica y profesional.

Finalmente, agradezco de manera especial a mi padre, por su sacrificio, confianza, comprensión y apoyo permanente. Sus consejos, palabras de aliento y ejemplo de perseverancia fueron pilares fundamentales para seguir adelante y alcanzar esta meta tan importante en mi vida.

INDICE

	Pág.
CAPÍTULO I: INTRODUCCIÓN	1
1.1 INTRODUCCIÓN.....	1
1.2 DIAGNÓSTICO	2
1.3 PLANTEAMIENTO Y FORMULACIÓN DEL PROBLEMA	3
1.3.1 Planteamiento del Problema:	6
1.3.2 Formulación del Problema:	6
1.4 PLANTEAMIENTO DE OBJETIVOS	6
1.4.1 Objetivo General.....	6
1.4.2 Objetivos Específicos.....	7
1.5 JUSTIFICACIÓN	7
1.5.1 Justificación Tecnológica.....	7
1.5.2 Justificación Social.....	8
1.5.3 Justificación Económica.....	8
1.6 ALCANCES Y DELIMITACIONES.....	8
1.6.1 Alcances	8
1.6.2 Delimitaciones	9
CAPÍTULO II: MARCO TEÓRICO CONCEPTUAL.....	10
2.1 METODOLOGÍA DE INVESTIGACIÓN MIXTA.....	10
2.1.1 Definición.....	10
2.1.2 Tipos de Enfoque de Investigación	12
2.2 METODOLOGÍA DE DESARROLLO: SCRUM.....	15
2.2.1 Roles y Artefactos en Scrum.....	16
2.2.2 Etapas del ciclo de Vida en Scrum.....	17
2.2.3 Historias de Usuario.....	19
2.3 METODOLOGIA DE MODELADO.....	20
2.3.1 Requerimientos Funcionales y No Funcionales.....	20
2.3.2 UML.....	22
2.3.3 Modelado de la Base de Datos NoSQL Relacional	27
2.4 HERRAMIENTAS Y TECNOLOGÍAS DE IMPLEMENTACIÓN.....	30
2.4.1 Lenguajes de Programación	30
2.4.2 Framework de Desarrollo Móvil.....	Error! Bookmark not defined.
2.4.3 Servicios Backend y Almacenamiento.....	Error! Bookmark not defined.

<i>2.4.4 Análisis Facial e Inteligencia Computacional.....</i>	Error! Bookmark not defined.
<i>2.4.5 Entornos y Herramientas de Desarrollo</i>	33
<i>2.4.6 Control de Versiones.....</i>	38
<i>2.4.7 Herramientas de Pruebas</i>	39
<i>2.4.8 Administración del Proyecto.....</i>	41

RESUMEN

El presente proyecto tiene como propósito desarrollar una Aplicación Móvil Inteligente de Apoyo a la Toma de Decisiones para la Recomendación de Monturas y Características de Lentes Ópticos en la Óptica OCULIST, ubicada en el municipio de Sacaba, departamento de Cochabamba. Actualmente, el proceso de recomendación se realiza de forma manual y depende principalmente de la experiencia y criterio del personal, lo que puede generar diferencias en las sugerencias, mayor tiempo de atención y dificultades para relacionar las características faciales del cliente con las monturas disponibles y los requerimientos técnicos de los lentes ópticos.

La aplicación propuesta permitirá registrar clientes, capturar o cargar imágenes faciales, analizar las proporciones y características generales del rostro, clasificar referencialmente su forma y generar recomendaciones personalizadas de monturas considerando aspectos como el tamaño, color, estilo y forma. Asimismo, integrará información óptica proporcionada por el especialista para recomendar características referenciales de los lentes, como el material, índice de refracción y tratamientos complementarios, sin reemplazar el diagnóstico ni el criterio profesional del optometrista.

Para el desarrollo del sistema se empleará una metodología de investigación mixta, combinando técnicas cualitativas y cuantitativas para identificar las necesidades de la óptica y los criterios de recomendación. El desarrollo del software se organizará mediante la metodología ágil Scrum, permitiendo avanzar por iteraciones, realizar pruebas continuas y mejorar progresivamente las funcionalidades de la aplicación.

La solución será desarrollada utilizando Flutter y Dart para la aplicación móvil, Firebase Authentication para el control de acceso, Cloud Firestore para la gestión de información, Firebase Storage para el almacenamiento de imágenes, Google ML Kit y MediaPipe Face Mesh para la detección y análisis de características faciales. Además, el sistema contará con un catálogo digital de monturas, historial de recomendaciones y un módulo administrativo para gestionar usuarios, monturas y reglas de recomendación.

En el ámbito tecnológico, el proyecto permitirá modernizar y sistematizar el proceso de asesoramiento de la Óptica OCULIST. En el ámbito social, ayudará a los clientes a comprender mejor qué monturas y características de lentes se adaptan a sus necesidades visuales, faciales y estéticas. En el ámbito económico, contribuirá a reducir el tiempo de atención, disminuir errores en la recomendación y fortalecer la competitividad de la óptica mediante un servicio más innovador, personalizado y eficiente.

CAPÍTULO I: INTRODUCCIÓN

1.1 INTRODUCCIÓN

En los últimos años, el avance de las tecnologías móviles, la visión por computadora y la inteligencia artificial ha transformado la forma en que las empresas interactúan con sus clientes, permitiendo ofrecer servicios más personalizados, eficientes e innovadores. Diversos estudios indican que los sistemas de recomendación inteligentes contribuyen a mejorar la experiencia de compra, incrementar la satisfacción de los usuarios y optimizar los procesos de atención en diferentes sectores comerciales.

Paralelamente, el mercado óptico ha experimentado un crecimiento constante debido al aumento de problemas visuales asociados al uso prolongado de dispositivos digitales, el envejecimiento de la población y una mayor conciencia sobre la salud visual. Este crecimiento ha impulsado a las ópticas a buscar nuevas herramientas tecnológicas que permitan diferenciar sus servicios y brindar una atención más personalizada a sus clientes.

En el ámbito de la óptica, la selección de monturas y lentes continúa siendo un proceso que depende en gran medida de la experiencia del asesor, la percepción estética del cliente y la interpretación de criterios técnicos relacionados con la prescripción óptica. Sin embargo, muchos usuarios desconocen qué tipo de montura se adapta mejor a la forma de su rostro o qué características del lente son las más adecuadas para sus necesidades visuales, lo que puede generar incertidumbre durante el proceso de compra.

La elección de una montura adecuada no solo depende de las preferencias personales del cliente, sino también de factores como la forma del rostro, las proporciones faciales, el tamaño de la montura, el color, el estilo y la compatibilidad con los lentes requeridos. Asimismo, cuando existe una prescripción óptica, deben considerarse aspectos técnicos como la graduación, el índice de refracción, el material del lente, los tratamientos especiales y el grosor estimado del cristal.

Ante esta situación, surge la necesidad de incorporar herramientas tecnológicas que apoyen el proceso de asesoramiento óptico mediante la automatización de ciertos criterios de análisis y recomendación. En este contexto, se propone el desarrollo de una aplicación móvil inteligente para la óptica "OCULIST", capaz de capturar o cargar imágenes del rostro del usuario, analizar características faciales mediante técnicas de visión por computadora y generar recomendaciones personalizadas de monturas y características de lentes ópticos.

La aplicación actuará como una herramienta de apoyo a la toma de decisiones, integrando análisis facial, reglas de recomendación estética y criterios técnicos ópticos para brindar sugerencias más rápidas, objetivas y personalizadas. De esta manera, se busca mejorar la experiencia del cliente, optimizar el proceso de atención y contribuir a la modernización de los servicios ofrecidos por la óptica "OCULIST", sin reemplazar el criterio profesional del optometrista o especialista del área.

1.2 DIAGNÓSTICO

La Óptica OCULIST, ubicada en el municipio de Sacaba del departamento de Cochabamba, brinda servicios de comercialización de monturas, lentes ópticos y asesoramiento a los clientes para la selección de productos que se adapten a sus necesidades visuales y estéticas. Sin embargo, el proceso de recomendación de monturas y de las características de los lentes ópticos se realiza de forma manual, basándose principalmente en la experiencia y criterio del personal. Las decisiones se toman mediante la observación visual y la comunicación con el cliente, sin contar con una herramienta tecnológica que apoye el análisis de las características faciales y facilite la selección de la alternativa más adecuada. Esta situación ha dado lugar a las siguientes problemáticas:

- La recomendación de monturas depende de la experiencia y criterio del personal, lo que puede generar diferencias en las sugerencias realizadas y afectar la uniformidad del proceso de atención.
- La determinación de las características de los lentes ópticos se realiza de manera manual, considerando la información proporcionada por la receta oftalmológica y la experiencia del asesor, lo que dificulta ofrecer recomendaciones más personalizadas de acuerdo con las necesidades visuales y preferencias del cliente.
- La ausencia de una aplicación móvil inteligente de apoyo a la toma de decisiones limita el análisis conjunto de la información del cliente, como las características faciales, el tipo de montura y las propiedades de los lentes ópticos, reduciendo la objetividad y precisión durante el proceso de recomendación.
- El proceso de selección requiere mayor tiempo de atención, debido a que el cliente debe probar diferentes monturas hasta encontrar una opción adecuada, incrementando el tiempo necesario para la toma de decisiones.
- La falta de incorporación de tecnologías de inteligencia artificial y visión artificial limita la modernización del servicio, restringiendo la posibilidad de ofrecer recomendaciones inteligentes, personalizadas y acordes con las tendencias tecnológicas actuales del sector óptico.

Como consecuencia de esta situación, se evidencia la necesidad de desarrollar una Aplicación Móvil Inteligente de Apoyo a la Toma de Decisiones para la Recomendación de Monturas y Características de Lentes Ópticos en la Óptica OCULIST, que contribuya a optimizar el proceso de recomendación, mejorar la calidad de la atención y apoyar al personal en la toma de decisiones mediante el uso de tecnologías de inteligencia artificial y visión artificial.

1.3 PLANTEAMIENTO Y FORMULACIÓN DEL PROBLEMA

1.3.1 Planteamiento del Problema:

La óptica OCULIST brinda asesoramiento a sus clientes para la selección de monturas y características de lentes ópticos. Este proceso requiere considerar distintos factores, como la forma y proporciones del rostro, el tamaño, color y estilo de la montura, además de aspectos técnicos relacionados con la graduación, el material, el índice de refracción y los tratamientos complementarios del lente.

Actualmente, la recomendación se realiza principalmente a partir de la experiencia y el criterio del personal de atención. Aunque este método permite orientar al cliente, puede generar diferencias en los criterios utilizados, debido a que no se dispone de una herramienta tecnológica que analice de manera sistemática las características faciales y relacione dicha información con criterios estéticos y parámetros ópticos.

Asimismo, los clientes suelen presentar dificultades para identificar qué monturas se adaptan mejor a la forma de su rostro y qué características de lentes son apropiadas para sus necesidades visuales. Esto ocasiona indecisión, necesidad de probar varias alternativas, desconocimiento de la información técnica y mayor tiempo durante el proceso de atención.

Por otra parte, la información de los usuarios y el catálogo de monturas no se encuentran completamente organizados y clasificados según criterios como forma del rostro, tamaño, color, estilo y compatibilidad con los diferentes tipos de lentes. Esta situación dificulta que el personal pueda relacionar rápidamente las características particulares de cada cliente con las opciones disponibles en la óptica.

Aunque existen aplicaciones internacionales para la prueba virtual de monturas, estas se concentran principalmente en la visualización estética y no integran completamente el análisis facial con la recomendación de características técnicas de lentes ópticos. Por ello,

la óptica OCULIST requiere una solución adaptada a su proceso de atención y a las necesidades de sus clientes.

CAUSAS

- Proceso de recomendación manual y subjetivo
 - Las mediciones y observaciones faciales se realizan de manera manual y no sistemática.
 - Las recomendaciones dependen principalmente de la experiencia y criterio individual del personal.
 - No existen reglas estandarizadas que relacionen las características faciales con los criterios ópticos.
 - Pueden presentarse diferencias en las recomendaciones realizadas por distintos asesores.
- Gestión desorganizada de la información
 - La información de los clientes puede registrarse de manera incompleta o desorganizada.
 - No existe una gestión centralizada de las características faciales, preferencias y datos ópticos del usuario.
 - Las recomendaciones anteriores no se encuentran registradas en un historial organizado.
 - El acceso a la información necesaria durante la atención puede demandar mayor tiempo.
- Catálogo de monturas sin clasificación especializada
 - Las monturas no se encuentran completamente clasificadas según la forma del rostro.
 - No existe una organización detallada por tamaño, color, estilo y características de cada montura.
 - No se identifica rápidamente la compatibilidad entre una montura y el tipo de lente requerido.
 - La variedad de modelos disponibles dificulta seleccionar las alternativas más apropiadas para cada cliente.
- Limitada estandarización de los criterios de asesoramiento

- El personal no dispone de una guía digital que reúna los criterios estéticos y ópticos de recomendación.
- La información técnica de los lentes puede resultar difícil de explicar y comparar durante la atención.
- No existe un mecanismo uniforme para aplicar los criterios de recomendación.
- El proceso depende del conocimiento y experiencia previa de cada asesor.
- Ausencia de una herramienta tecnológica de apoyo
 - No se cuenta con una aplicación móvil que capture y analice las características faciales del usuario.
 - No existe un motor de recomendación que relacione la forma del rostro con las monturas disponibles.
 - No se dispone de una herramienta que integre el análisis facial con parámetros ópticos.
 - Las decisiones se realizan sin un apoyo tecnológico objetivo y sistematizado.

EFFECTOS

- Recomendaciones subjetivas y poco estandarizadas
 - Las sugerencias pueden variar según el criterio del asesor que atiende al cliente.
 - No siempre se aplican los mismos criterios faciales, estéticos y ópticos.
 - Se dificulta garantizar uniformidad en la calidad del asesoramiento.
 - Las recomendaciones pueden no considerar de manera conjunta todos los factores necesarios.
- Dificultad para identificar monturas adecuadas
 - Los clientes deben probar varias monturas antes de seleccionar una opción.
 - Se incrementa la indecisión durante el proceso de elección.
 - Puede seleccionarse una montura que no armonice adecuadamente con la forma del rostro.
 - Las opciones disponibles no se filtran rápidamente según las características del usuario.
- Riesgo de incompatibilidad entre la montura y el lente
 - La montura seleccionada puede no ser apropiada para el tipo o grosor referencial del lente.

- Pueden presentarse dificultades relacionadas con la graduación o el material recomendado.
- Se requiere una evaluación adicional para corregir o confirmar la selección.
- La recomendación estética puede realizarse sin considerar completamente las necesidades ópticas.
- Mayor tiempo de atención
 - El personal debe evaluar manualmente diferentes alternativas.
 - La búsqueda de monturas adecuadas puede prolongar el proceso de asesoramiento.
 - La falta de información organizada dificulta responder rápidamente a las necesidades del cliente.
 - Se reduce la eficiencia en la atención brindada por la óptica.
- Decisiones con información técnica limitada
 - Los clientes pueden desconocer las diferencias entre materiales, tratamientos e índices de refracción.
 - La elección puede basarse principalmente en la apariencia de la montura.
 - Se genera incertidumbre sobre cuál opción se adapta mejor a las necesidades visuales.
 - Puede disminuir la confianza del cliente durante la toma de decisiones.
- Atención menos moderna e interactiva
 - La óptica aprovecha de manera limitada las tecnologías de análisis facial y recomendación inteligente.
 - El proceso de atención ofrece menor interacción digital con el cliente.
 - Se dificulta diferenciar el servicio de la óptica frente a otras alternativas del mercado.
 - Se limita la modernización y personalización del asesoramiento óptico.

1.3.2 Planteamiento del Problema:

¿Cómo mejorar la recomendación personalizada de monturas y características de lentes ópticos a los usuarios de la óptica "OCULIST"?

1.4 PLANTEAMIENTO DE OBJETIVOS

1.4.1 Objetivo General

Desarrollar una aplicación móvil inteligente basada en análisis facial y parámetros ópticos que apoye la recomendación personalizada de monturas y características de lentes ópticos para los clientes de la óptica OCULIST.

1.4.2 Objetivos Específicos

- Identificar las variables faciales, estéticas y ópticas necesarias para la recomendación personalizada de monturas y lentes.
- Diseñar una base de datos para gestionar clientes, monturas, características de lentes, reglas de recomendación e historial de consultas.
- Diseñar un módulo de captura y análisis facial que permita detectar rasgos generales del rostro del usuario.
- Implementar un motor de recomendación basado en reglas que relacione forma del rostro, color, tamaño de montura y parámetros ópticos.
- Implementar un módulo que permita registrar clientes, capturar imagen facial, ingresar datos ópticos y mostrar recomendaciones.
- Validar el funcionamiento del sistema mediante pruebas funcionales, pruebas de usabilidad y comparación de recomendaciones con criterios de especialistas del área óptica.
- Diseñar y elaborar un manual de usuario que facilite la correcta utilización del sistema por parte del personal de la óptica.

1.5 JUSTIFICACIÓN

1.5.1 Justificación Tecnológica

La propuesta se justifica técnicamente debido a la necesidad de incorporar herramientas tecnológicas que apoyen el proceso de recomendación de monturas y características de lentes ópticos mediante el análisis de información facial y parámetros ópticos. La aplicación permitirá optimizar el proceso de asesoramiento, proporcionando recomendaciones más rápidas, objetivas y personalizadas para los clientes de la óptica "OCULIST".

El sistema será desarrollado bajo una arquitectura cliente-servidor, donde la aplicación móvil actuará como cliente para la interacción con los usuarios, mientras que el servidor gestionará el almacenamiento, procesamiento y administración de la información. Esta arquitectura facilitará la escalabilidad, el mantenimiento y la disponibilidad de los datos.

Asimismo, se implementarán tecnologías de visión por computadora para el procesamiento de imágenes faciales, permitiendo identificar características relevantes del rostro que servirán como base para la recomendación de monturas adecuadas. Complementariamente, se aplicarán reglas de negocio definidas por especialistas ópticos para sugerir características técnicas de los lentes de acuerdo con las necesidades visuales del usuario.

1.5.2 Justificación Social

El proyecto beneficia a usuarios que requieren lentes ópticos, ya que apoya la toma de decisiones en la selección de una montura acorde a sus características y preferencias, permitiéndoles comprender mejor qué montura y características de lente pueden adaptarse a su rostro, estilo y necesidad visual. También promueve el uso de herramientas digitales en servicios relacionados con salud visual, sin reemplazar la orientación profesional.

1.5.3 Justificación Económica

El desarrollo de la aplicación móvil permitirá optimizar el proceso de recomendación de monturas y lentes ópticos, reduciendo el tiempo de atención mediante la automatización de tareas de análisis y asesoramiento. Asimismo, la recomendación asistida contribuirá a disminuir errores en la selección de productos, mejorando la precisión de las sugerencias ofrecidas a los clientes.

La personalización de las recomendaciones incrementará la satisfacción y confianza de los usuarios durante el proceso de compra, favoreciendo una mejor experiencia de atención. Además, la incorporación de tecnologías de análisis facial y apoyo a la toma de decisiones permitirá a la óptica "OCULIST" diferenciarse de la competencia mediante la oferta de servicios innovadores.

Concepto	Beneficio
Automatización	Reduce el tiempo de atención al cliente.
Recomendación asistida	Disminuye errores en la selección de monturas y lentes.
Atención personalizada	Incrementa la satisfacción y experiencia del cliente.
Diferenciación tecnológica	Mejora la competitividad de la óptica.

Por lo tanto, el proyecto es económicamente viable, ya que contribuye a optimizar recursos, mejorar la calidad del servicio y fortalecer la posición competitiva de la óptica "OCULIST".

1.6 ALCANCES Y DELIMITACIONES

1.6.1 Alcances

Los principales módulos del sistema son:

- Registro de clientes atendidos en la óptica, para llevar un control organizado de la información de los usuarios.
- Captura o carga de imagen del rostro del usuario, para analizar sus características faciales.

- Detección básica del rostro y análisis de proporciones faciales.
- Clasificación referencial de la forma del rostro: ovalado, redondo, cuadrado, alargado, triangular o corazón.
- Recomendación de forma, montura, color, tamaño y estilo.
- Catálogo digital de monturas disponibles en la óptica.
- Historial de recomendaciones generadas.
- Panel administrativo básico para gestionar monturas, lentes y reglas de recomendación.

1.6.2 Delimitaciones

- Para mantener un enfoque realista y garantizar la implementación efectiva del sistema en esta etapa, se definen las siguientes delimitaciones:
- El sistema no reemplazará el diagnóstico ni la evaluación profesional del optometrista.
- La precisión del análisis facial dependerá de la calidad de la imagen, iluminación y posición del rostro.
- El grosor del lente será referencial y deberá ser confirmado por el especialista o laboratorio óptico.
- La primera versión no incluirá realidad aumentada avanzada ni prueba 3D completa de monturas.
- La recomendación estará basada en reglas previamente definidas y validadas con expertos.
- El sistema requerirá un catálogo previamente cargado para mostrar productos reales disponibles.
- El sistema no generará ni calculará parámetros ópticos como esfera, cilindro, eje o adición, ya que estos son determinados por el especialista en el laboratorio óptico.

CAPÍTULO II: MARCO TEÓRICO CONCEPTUAL

2.1 Metodología de Investigación Mixta

2.1.1 Definición

La metodología de investigación mixta es un enfoque que combina dos tipos de investigación: cualitativa y cuantitativa. Su finalidad es obtener una visión más completa del problema de estudio, utilizando tanto información basada en opiniones, experiencias y observaciones como datos numéricos que pueden ser medidos y analizados. (Creswell, 2018)

En el desarrollo de un proyecto, este enfoque permite conocer las necesidades y dificultades de los usuarios mediante entrevistas u observaciones, y al mismo tiempo obtener información cuantificable mediante encuestas o cuestionarios. De esta manera, se pueden analizar los resultados desde diferentes perspectivas y obtener conclusiones más completas. (Hernández-Sampieri, 2018)

Características principales

- Combina métodos cualitativos y cuantitativos: Integra información obtenida mediante entrevistas, observaciones y opiniones con datos numéricos provenientes de encuestas y cuestionarios.
- Permite analizar un problema desde diferentes perspectivas: Facilita el estudio de un fenómeno considerando tanto la información subjetiva de los participantes como los resultados medibles.
- Utiliza diferentes técnicas de recolección de información: Entre las técnicas más utilizadas se encuentran las encuestas, entrevistas, observación y cuestionarios.
- Permite obtener datos numéricos y opiniones de los participantes: Los datos cuantitativos permiten identificar tendencias y resultados medibles, mientras que los cualitativos ayudan a comprender las experiencias y percepciones de las personas.
- Facilita una comprensión más completa del problema: La integración de ambos enfoques proporciona una visión más amplia y profunda del objeto de estudio.
- Ayuda a comparar y complementar los resultados obtenidos: Los resultados de un método pueden complementar o contrastarse con los resultados obtenidos mediante el otro método.

- Permite conocer qué sucede y por qué sucede: Los datos cuantitativos ayudan a identificar qué está ocurriendo, mientras que los datos cualitativos permiten comprender las razones, experiencias y opiniones relacionadas con el problema de estudio.

Ventajas

- Mayor comprensión del problema: Permite analizar el problema desde diferentes puntos de vista.
- Complementación de resultados: Los datos cualitativos y cuantitativos se complementan entre sí.
- Mayor profundidad en la investigación: Permite conocer no solamente los resultados, sino también las razones y experiencias relacionadas con el problema.
- Obtención de información más completa: Combina datos numéricos con información descriptiva y opiniones de los participantes.
- Mayor capacidad para tomar decisiones: Los resultados obtenidos pueden proporcionar información útil para plantear soluciones más adecuadas.
- Permite validar los resultados: La información obtenida mediante un método puede compararse con los resultados del otro método para encontrar coincidencias o diferencias.
- Flexibilidad: Puede adaptarse a diferentes tipos de problemas y proyectos de investigación.

Desventajas y limitaciones

- Requiere más tiempo: La aplicación de encuestas, entrevistas y observaciones puede aumentar el tiempo necesario para realizar la investigación.
- Mayor cantidad de recursos: Puede requerir más recursos humanos, económicos y materiales.
- Mayor complejidad en el análisis: El investigador debe analizar e interpretar datos cualitativos y cuantitativos.
- Requiere conocimientos de diferentes métodos: Es necesario conocer técnicas de análisis estadístico y técnicas de análisis cualitativo.
- Puede ser difícil integrar los resultados: En algunos casos, los resultados cualitativos y cuantitativos pueden presentar diferencias o contradicciones.

2.1.2 Tipos de Enfoque de Investigación

2.1.2.1 Enfoque cualitativo

El enfoque cualitativo busca comprender fenómenos a partir de los significados, experiencias, opiniones, comportamientos e interacciones de las personas. Trabaja principalmente con información textual, verbal, visual o contextual y procura interpretar el problema dentro del ambiente en el que ocurre, sin reducirlo únicamente a valores numéricos (Hernández-Sampieri y Mendoza, 2018).

Características

- Analiza opiniones, experiencias y percepciones.
- Utiliza entrevistas y observación como técnicas principales.
- Busca comprender el problema desde la perspectiva de los participantes.
- Trabaja principalmente con información descriptiva.
- Permite analizar situaciones en su contexto natural.
- Tiene un proceso flexible que puede adaptarse durante la investigación.
- Busca comprender las causas y características de un problema.

Ventajas

- Permite conocer las opiniones y experiencias de los participantes.
- Proporciona información detallada y profunda.
- Ayuda a comprender las causas de un problema.
- Permite descubrir necesidades que pueden no ser evidentes inicialmente.
- Es flexible y puede adaptarse a diferentes situaciones.
- Facilita el análisis de comportamientos y experiencias.

Desventajas

- Los resultados pueden ser más difíciles de comparar.
- El análisis de la información puede requerir bastante tiempo.
- La interpretación puede depender del criterio del investigador.
- Generalmente trabaja con grupos pequeños de participantes.
- Los resultados no siempre pueden generalizarse a toda una población.

- Requiere habilidades para realizar entrevistas y analizar información.

2.1.2.2 Enfoque cuantitativo

El enfoque cuantitativo se basa en la medición de variables y en el análisis de datos numéricos mediante procedimientos estadísticos. Su propósito puede ser describir una situación, comparar grupos, identificar tendencias, comprobar relaciones o evaluar resultados de forma estructurada y verificable (Hernández-Sampieri y Mendoza, 2018).

La investigación cuantitativa requiere definir con anticipación las variables, los indicadores, la población, la muestra y el instrumento de medición. Para que los resultados sean confiables, las preguntas deben ser claras, las escalas deben aplicarse de manera uniforme y el análisis debe corresponder al tipo de datos obtenidos (Creswell y Plano Clark, 2018).

Características

- Utiliza datos numéricos y estadísticos.
- Busca medir variables de manera objetiva.
- Utiliza encuestas y cuestionarios como técnicas de recolección de datos.
- Permite representar los resultados mediante tablas y gráficos.
- Sigue un proceso estructurado y organizado.
- Busca identificar relaciones entre diferentes variables.
- Los resultados pueden ser comparados y analizados estadísticamente.

Ventajas

- Permite obtener resultados medibles y objetivos.
- Facilita la comparación de resultados.
- Permite analizar una gran cantidad de datos.
- Los resultados pueden representarse mediante gráficos y tablas.
- Facilita la identificación de tendencias y patrones.
- Permite realizar análisis estadísticos.

Desventajas

- Puede no explicar completamente las razones detrás de los resultados.
- Puede limitar las respuestas de los participantes a opciones establecidas.

- No siempre permite conocer las opiniones y experiencias personales con profundidad.
- Requiere conocimientos para realizar análisis estadísticos.
- La elaboración y aplicación de encuestas puede requerir tiempo y recursos.

2.1.2.3 Herramientas

Las herramientas de investigación son los recursos y medios utilizados por el investigador para recopilar, organizar y analizar la información necesaria durante el desarrollo de una investigación. Estas herramientas permiten obtener datos de manera ordenada y facilitar el estudio del problema planteado. Entre las herramientas más utilizadas se encuentran los cuestionarios, las entrevistas, las fichas de observación y los formularios, dependiendo del enfoque y los objetivos de la investigación.

2.1.2.3.1 Entrevista semiestructurada

La entrevista semiestructurada es una técnica de investigación que utiliza una guía de preguntas previamente preparada, pero permite al entrevistador realizar preguntas adicionales según las respuestas obtenidas durante la conversación. Esta técnica combina preguntas planificadas con la flexibilidad necesaria para profundizar en temas importantes que surjan durante la entrevista. Su objetivo es obtener información detallada sobre las opiniones, experiencias, necesidades y conocimientos de las personas entrevistadas.

Su finalidad es obtener información profunda sin perder la orientación del estudio. La guía debe contener preguntas abiertas, comprensibles y relacionadas con los objetivos de investigación. También es recomendable realizar una prueba piloto, registrar las respuestas con autorización y elaborar notas que ayuden a interpretar el contexto de la conversación (Kallio et al., 2016).

2.1.2.3.2 Análisis de documentación

El análisis documental es una técnica sistemática que permite revisar, seleccionar, interpretar y organizar información contenida en documentos. Las fuentes pueden ser impresas o digitales e incluir reglamentos, formularios, manuales, reportes, historiales, catálogos, registros, procedimientos institucionales, fotografías y otros materiales relacionados con el objeto de estudio (Bowen, 2009).

Esta técnica no consiste únicamente en leer documentos. Requiere establecer criterios de selección, comprobar su autenticidad y actualidad, identificar el propósito con el que fueron elaborados y extraer los datos relevantes mediante fichas, matrices o categorías de análisis.

Posteriormente, la información se compara para reconocer coincidencias, diferencias, vacíos, reglas y procesos documentados (Bowen, 2009).

2.1.2.3.3 Observación directa

La observación directa es una técnica que permite recopilar información al presenciar y registrar hechos, actividades, interacciones y condiciones en el lugar donde ocurren. Su propósito es conocer cómo se desarrolla realmente un proceso, qué participantes intervienen, qué recursos se utilizan y qué dificultades aparecen durante la ejecución de las tareas (Hernández-Sampieri y Mendoza, 2018).

La observación debe planificarse mediante categorías o indicadores y registrarse en una ficha, diario de campo o lista de verificación. Puede ser participante, cuando el investigador se integra en la actividad, o no participante, cuando observa sin intervenir. En ambos casos es necesario diferenciar los hechos observados de las interpretaciones personales (Hernández-Sampieri y Mendoza, 2018).

2.1.2.3.4 Cuestionario de usabilidad

El cuestionario de usabilidad es un instrumento compuesto por preguntas o afirmaciones que permiten conocer la percepción de los usuarios después de interactuar con un sistema. Puede evaluar facilidad de aprendizaje, comprensión, eficiencia, consistencia, confianza, satisfacción y dificultades encontradas durante la ejecución de tareas (ISO, 2018).

2.2 Metodología de Desarrollo: Scrum

Scrum es un marco de trabajo ágil para generar valor mediante soluciones adaptativas a problemas complejos. Organiza el trabajo en periodos de duración fija denominados Sprints, dentro de los cuales un equipo transforma elementos priorizados del Product Backlog en un incremento útil y verificable. Scrum no describe todas las técnicas de desarrollo que deben utilizarse; establece responsabilidades, eventos, artefactos y reglas mínimas que proporcionan estructura al proceso (Schwaber y Sutherland, 2020).

El marco se apoya en el empirismo, según el cual las decisiones se toman con base en lo observado y aprendido durante el trabajo. Sus tres pilares son transparencia, inspección y adaptación. La transparencia hace visible el estado real del producto y del trabajo; la inspección permite revisar frecuentemente los resultados; y la adaptación introduce cambios cuando se detectan desviaciones o nuevas necesidades (Schwaber y Sutherland, 2020).

2.2.1 Roles y Artefactos en Scrum

En Scrum existen diferentes roles, que actualmente se denominan responsabilidades dentro del equipo Scrum, y artefactos que permiten organizar, controlar y visualizar el trabajo realizado durante el desarrollo del proyecto. Estos elementos ayudan al equipo a mantener una comunicación clara y trabajar de manera coordinada para alcanzar el objetivo del producto.

Roles en Scrum:

- **Product Owner:** maximiza el valor del producto y gestiona de manera efectiva el Product Backlog. Define y comunica el Objetivo del Producto, ordena los elementos y procura que el trabajo sea visible y comprendido.
- **Scrum Master:** ayuda a que Scrum sea comprendido y aplicado. Facilita la mejora del equipo, contribuye a eliminar impedimentos y apoya la colaboración con la organización y los interesados.
- **Developers:** son las personas que crean el incremento. Elaboran el plan del Sprint, mantienen la calidad mediante la Definición de Terminado y adaptan diariamente su trabajo para alcanzar el Objetivo del Sprint.

Artefactos en Scrum:

- **Product Backlog** es una lista ordenada de todos los elementos que podrían ser necesarios para mejorar el producto. Incluye funcionalidades, mejoras, correcciones y otros trabajos necesarios para alcanzar el objetivo del producto.
- **Sprint Backlog** contiene los elementos seleccionados del Product Backlog que el equipo desarrollará durante un Sprint. También incluye el plan de trabajo necesario para alcanzar el Sprint Goal.
- **Increment** es el resultado funcional y utilizable del trabajo realizado durante un Sprint. Representa una versión mejorada del producto que cumple con la Definition of Done (Definición de Terminado).

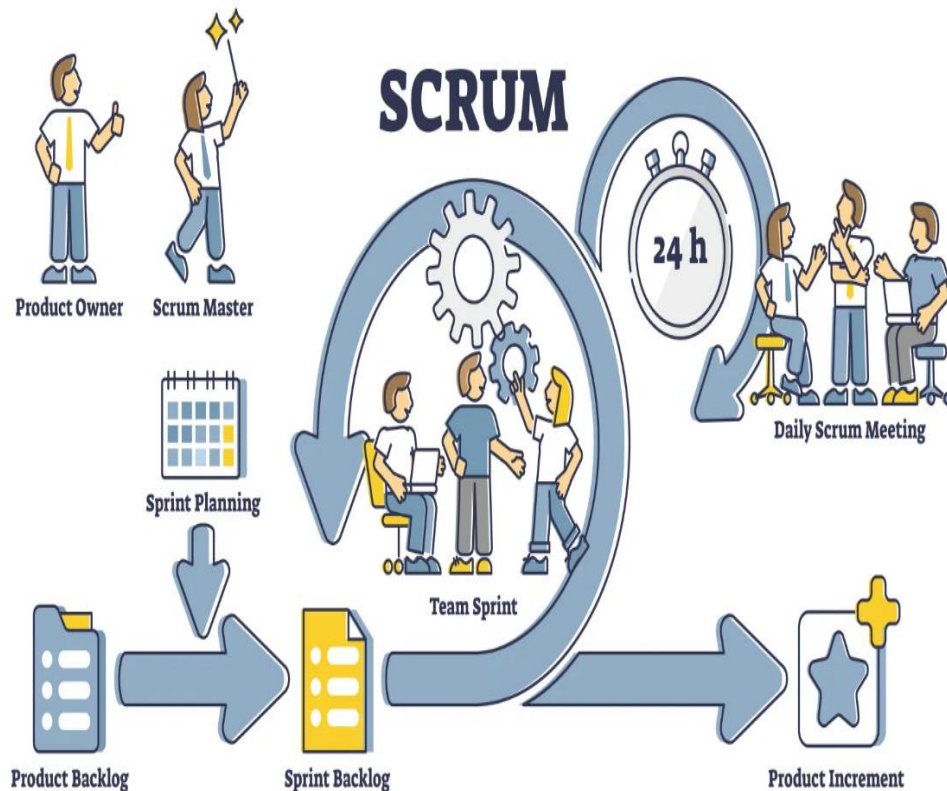


Figura 1: Metodología Scrum

Fuente: Tomado de ¿Qué es y cómo funciona SCRUM?, ILUNION (2025), <https://www.ilunion.com/es/blog-puntoilunion/scrums>

2.2.2 Etapas del ciclo de Vida en Scrum

2.2.2.1 Inicio

La etapa de inicio establece la dirección general antes de comenzar los Sprints. Comprende la identificación del problema, los usuarios, los interesados, las necesidades principales, las restricciones y el valor que se espera obtener. A partir de esta información se formula una visión del producto y un Objetivo del Producto suficientemente claros para orientar las decisiones posteriores (Schwaber y Sutherland, 2020).

Durante el inicio también se conforma el Scrum Team, se aclaran las responsabilidades y se construye una primera versión del Product Backlog. Este backlog inicial no necesita contener todos los detalles; debe reunir los resultados, funciones, mejoras y riesgos conocidos, ordenados de manera preliminar. Los elementos se refinan progresivamente a medida que aumenta el conocimiento sobre el producto y sus usuarios (Schwaber y Sutherland, 2020).

Un inicio adecuado evita comenzar el desarrollo con objetivos contradictorios o un alcance indefinido. No pretende congelar todos los requisitos, sino establecer una base común que permita iniciar el primer Sprint, evaluar resultados tempranos y adaptar las prioridades según la evidencia obtenida.

Actividades habituales

- Definir la visión, el objetivo y los límites iniciales del producto.
- Identificar usuarios, interesados, restricciones y riesgos principales.
- Asignar las responsabilidades del Scrum Team.
- Crear y ordenar una versión inicial del Product Backlog.
- Acordar criterios generales de calidad y una Definición de Terminado inicial.

2.2.2.2 Planeación y Estimación

La planificación transforma los objetivos generales en trabajo realizable. Durante el refinamiento del Product Backlog, los elementos se aclaran, dividen y ordenan; durante la Sprint Planning, el equipo selecciona el trabajo que puede completar, define el Objetivo del Sprint y construye un plan para producir el incremento (Schwaber y Sutherland, 2020).

La estimación expresa el esfuerzo, la complejidad o la incertidumbre relativa de los elementos. Puede realizarse mediante puntos de historia, tallas o técnicas de comparación. La estimación no debe interpretarse como una promesa exacta de horas; su función es apoyar la selección de trabajo, reconocer riesgos y mejorar la previsibilidad a partir de la experiencia de Sprints anteriores.

2.2.2.3 Implementación

La implementación comprende el análisis detallado, diseño, programación, integración, documentación y pruebas necesarias para convertir los elementos seleccionados en un incremento. Los Developers administran el Sprint Backlog y adaptan el plan cuando descubren nueva información, siempre que no pongan en riesgo el Objetivo del Sprint (Schwaber y Sutherland, 2020).

La Daily Scrum permite inspeccionar diariamente el progreso y ajustar el trabajo inmediato. Además de este evento, el equipo puede comunicarse durante el día, revisar código, ejecutar pruebas y resolver impedimentos. La calidad no se deja para el final del Sprint: cada elemento debe cumplir los criterios acordados y la Definición de Terminado antes de considerarse parte del incremento.

2.2.2.4 Revisión y Retrospectiva

Al finalizar el Sprint se realizará una revisión del incremento con los interesados. La demostración deberá utilizar casos reales o controlados y verificar los criterios de aceptación. Las observaciones podrán originar cambios en la prioridad, nuevas historias o ajustes sobre funciones ya desarrolladas (Schwaber y Sutherland, 2020).

Posteriormente se efectuará una retrospectiva para analizar qué funcionó, qué dificultades aparecieron y qué acciones pueden mejorar el siguiente Sprint. En un proyecto individual, esta reflexión puede registrarse mediante una ficha breve con problemas técnicos, desviaciones de tiempo, decisiones adoptadas y lecciones aprendidas.

2.2.2.5 Lanzamiento

La liberación consiste en poner un incremento a disposición de usuarios o interesados cuando cumple los criterios técnicos, funcionales y de calidad definidos. Scrum permite liberar valor antes de finalizar un Sprint, siempre que el incremento cumpla la Definición de Terminado; por tanto, el lanzamiento no es un evento obligatorio separado ni debe esperar necesariamente al cierre de todo el proyecto (Schwaber y Sutherland, 2020).

Antes de una liberación se revisan instalación, configuración, datos, permisos, seguridad, compatibilidad, respaldo, documentación y pruebas de aceptación. Después del lanzamiento se recopila retroalimentación y se registran incidencias, las cuales pueden originar nuevos elementos o cambios de prioridad en el Product Backlog. De esta manera, la entrega forma parte del ciclo de inspección y adaptación.

2.2.3 Historias de Usuario

Las historias de usuario son descripciones breves de una necesidad o capacidad expresadas desde la perspectiva de quien obtiene valor. No constituyen especificaciones completas; funcionan como recordatorios para mantener una conversación y definir posteriormente los detalles, reglas y criterios de aceptación necesarios (Cohn, 2004).

Una forma habitual de redacción es: “Como [tipo de usuario], quiero [capacidad], para [beneficio]”. Esta estructura identifica al usuario, la acción esperada y el valor asociado. Cada historia debe acompañarse de criterios de aceptación observables que indiquen las condiciones que deben cumplirse para considerarla terminada y facilitar su verificación (Cohn, 2004).

2.3 Metodología de Modelado

El modelado de software representa de manera simplificada los requisitos, procesos, estructura, comportamiento y datos de un sistema. Los modelos ayudan a comprender el problema antes de programar, comunicar decisiones entre participantes y detectar inconsistencias que serían más costosas de corregir durante la implementación (Sommerville, 2016).

Un modelo no pretende incluir todos los detalles del sistema. Debe seleccionar la información relevante para su propósito y mantener relación con los requisitos que representa. Los diagramas, especificaciones y modelos de datos deben actualizarse cuando cambian las decisiones importantes, evitando que la documentación quede separada del producto desarrollado (Sommerville, 2016).

2.3.1 Requerimientos Funcionales y No Funcionales

Los requerimientos de software describen necesidades, capacidades, condiciones y restricciones que debe satisfacer un sistema. La ingeniería de requisitos comprende actividades de obtención, análisis, especificación, validación y gestión de cambios. Un requerimiento de calidad debe ser necesario, claro, consistente, factible, verificable y trazable (ISO/IEC/IEEE, 2018).

La clasificación más utilizada distingue requerimientos funcionales y no funcionales. Los primeros expresan servicios o comportamientos; los segundos establecen atributos de calidad, restricciones y condiciones de operación. Ambos tipos son necesarios, porque un sistema puede ejecutar las funciones solicitadas y aun así resultar inseguro, lento, difícil de utilizar o incompatible (Sommerville, 2016).

2.3.1.1 Requerimientos Funcionales

Los requerimientos funcionales especifican qué debe hacer el sistema frente a determinadas entradas, condiciones o acciones del usuario. Describen servicios, reglas de negocio, cálculos, validaciones, consultas, registros, modificaciones, notificaciones y respuestas esperadas. También pueden indicar comportamientos ante errores o situaciones excepcionales (ISO/IEC/IEEE, 2018).

Ventajas

- Permiten definir claramente las funciones que tendrá el sistema.
- Facilitan la planificación y organización del desarrollo.
- Ayudan a los desarrolladores a comprender las necesidades de los usuarios.

- Permiten verificar si el sistema cumple con los objetivos establecidos.
- Facilitan la realización de pruebas funcionales.
- Reducen la posibilidad de desarrollar funciones innecesarias.

Desventajas

- Si están mal definidos, pueden generar errores durante el desarrollo.
- Los cambios frecuentes en los requisitos pueden aumentar el tiempo de desarrollo.
- Una gran cantidad de requerimientos puede hacer que el proyecto sea más complejo.
- Si no se describen con claridad, pueden existir diferentes interpretaciones entre usuarios y desarrolladores.
- Pueden requerir modificaciones cuando cambian las necesidades del negocio.

2.3.1.2 Requerimientos No Funcionales

Los requerimientos no funcionales establecen atributos de calidad y restricciones que condicionan la forma en que se ofrecen las funciones. Pueden referirse a rendimiento, seguridad, usabilidad, confiabilidad, disponibilidad, mantenibilidad, portabilidad, compatibilidad, privacidad, accesibilidad y restricciones tecnológicas o normativas (ISO/IEC/IEEE, 2018).

Ventajas

- Permiten mejorar la calidad general del sistema.
- Ayudan a garantizar la seguridad de la información.
- Mejoran la experiencia y satisfacción de los usuarios.
- Permiten establecer niveles adecuados de rendimiento.
- Facilitan la evaluación de aspectos como velocidad y disponibilidad.
- Ayudan a crear sistemas más confiables y fáciles de mantener.

Desventajas

- Pueden ser difíciles de definir y medir correctamente.
- Algunos requieren recursos adicionales para su implementación.
- Pueden aumentar los costos y el tiempo de desarrollo.

- Requieren pruebas específicas para verificar su cumplimiento.
- Si no se establecen correctamente, pueden afectar la calidad del sistema.
- En algunos casos, mejorar una característica puede afectar otra, por ejemplo, aumentar la seguridad puede hacer más complejo el acceso.

2.3.2 UML

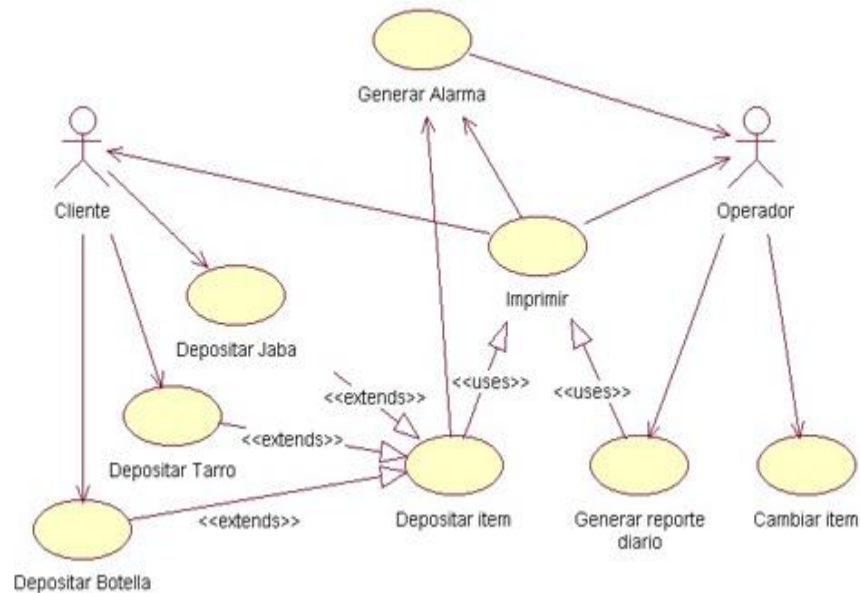


Figura 2: UML: Diagrama de Caso de Usos

Fuente: DocIRs. (2008.). *¿Qué es UML?*. Recuperado el 4 de agosto de 2026, de <https://docirs.cl/uml.asp>

El **Lenguaje Unificado de Modelado (UML)** es un lenguaje gráfico estandarizado para visualizar, especificar, construir y documentar elementos de sistemas de software. Proporciona diferentes tipos de diagramas para representar estructura, comportamiento e interacción, utilizando símbolos y relaciones con significado definido (Object Management Group, 2017).

UML no es una metodología de desarrollo ni obliga a crear todos sus diagramas. Cada proyecto debe seleccionar aquellos que aporten información útil. Los diagramas deben conservar un nivel de detalle coherente con su propósito y acompañarse de descripciones cuando la notación por sí sola pueda resultar ambigua (Object Management Group, 2017).

2.3.2.1 Casos de Uso

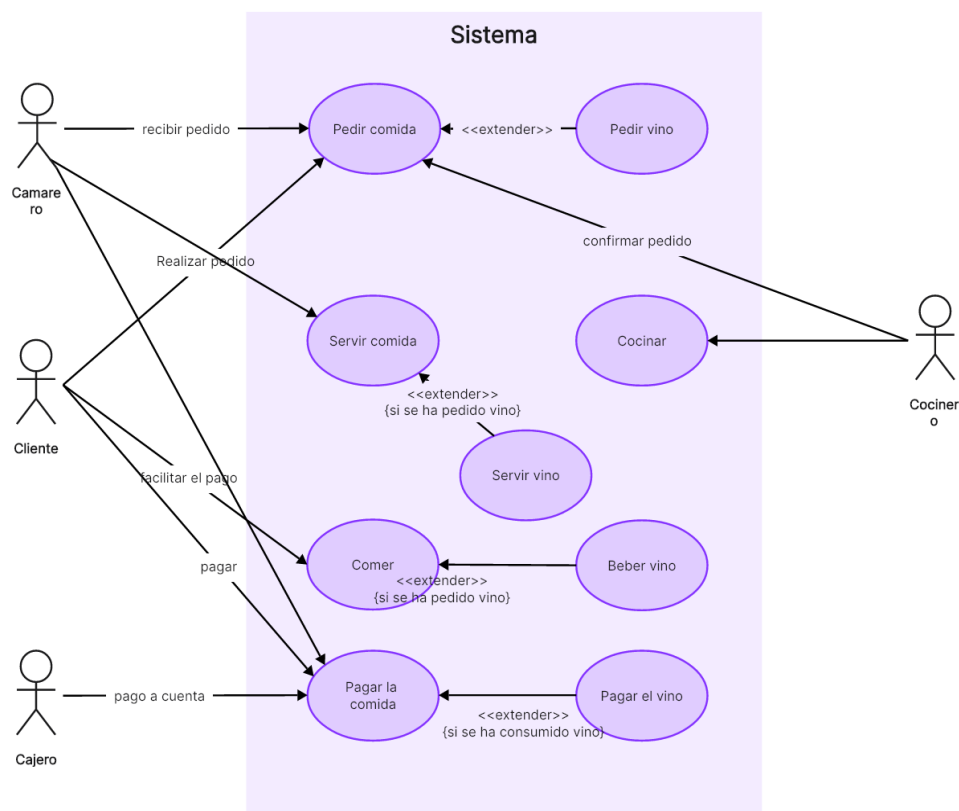


Figura 3: UML: Diagrama de Caso de Usos

Fuente: Boardmix. (2024.). *¿Qué es diagramas de casos de uso?*. Recuperado el 4 de agosto de 2026, de <https://boardmix.com/es/articles/use-case-examples/>

El **diagrama de casos de uso** representa los objetivos que actores externos alcanzan mediante la interacción con un sistema. Un actor puede ser una persona, otro sistema o un dispositivo que desempeña un rol. Los casos de uso describen capacidades visibles desde el exterior y se colocan dentro del límite del sistema (Object Management Group, 2017).

Sus elementos principales son actores, casos de uso, asociaciones y relaciones de inclusión, extensión o generalización. El diagrama muestra quién utiliza cada función y cuál es el alcance funcional, pero no presenta la secuencia detallada ni la implementación interna. Por esta razón debe complementarse con una descripción textual de flujos, precondiciones, poscondiciones y excepciones (Object Management Group, 2017).

Ventajas:

- Presenta el alcance funcional mediante una representación comprensible para usuarios técnicos y no técnicos.

- Facilita identificar actores, objetivos y relaciones entre funciones.
- Sirve como base para historias de usuario, escenarios y pruebas de aceptación.

Desventajas:

- No muestra la estructura interna del software ni los detalles de implementación.
- Puede resultar ambiguo cuando no se acompaña de una descripción textual.
- Un diagrama con demasiados casos de uso pierde claridad.

2.3.2.2 Diagrama de Clases

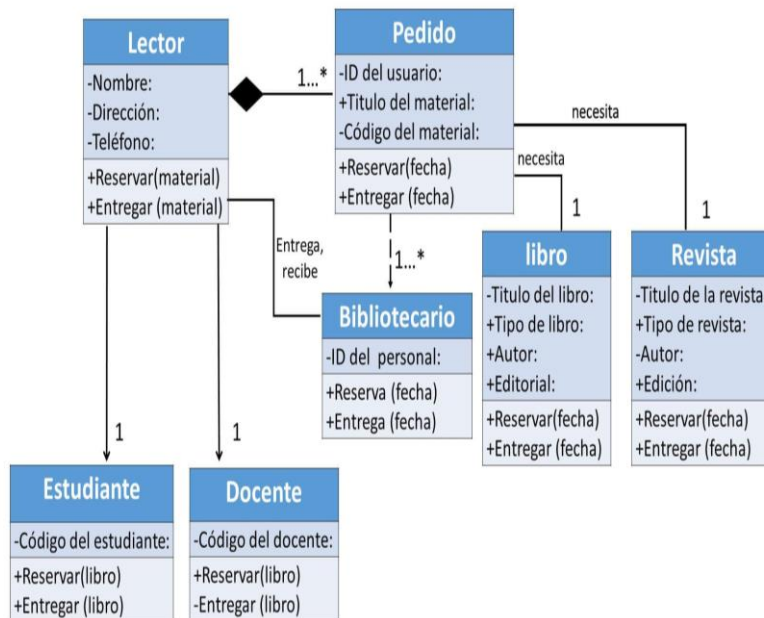


Figura 4: Diagrama de Clases

Fuente: Tomado de Ejemplos de Diagramas de Clases UML, WEByempresas(2022), <https://www.webyempresas.com/ejemplos-de-diagramas-de-clases-uml/>

El **diagrama de clases** representa la estructura estática de un sistema orientado a objetos. Muestra clases, atributos, operaciones y relaciones como asociación, agregación, composición, dependencia, generalización y realización. También puede expresar multiplicidades y visibilidad de miembros (Object Management Group, 2017).

Ventajas:

- Permite visualizar entidades, responsabilidades y relaciones antes de implementar.
- Favorece la organización del código y la reutilización de componentes.
- Ayuda a detectar dependencias y clases con responsabilidades excesivas.

Desventajas:

- Puede volverse complejo cuando se incluyen demasiados atributos y operaciones.
- El diseño inicial puede cambiar a medida que se comprenden mejor los requerimientos.
- No representa por sí solo el orden temporal de las interacciones.

2.3.2.3 Diagrama de Secuencia

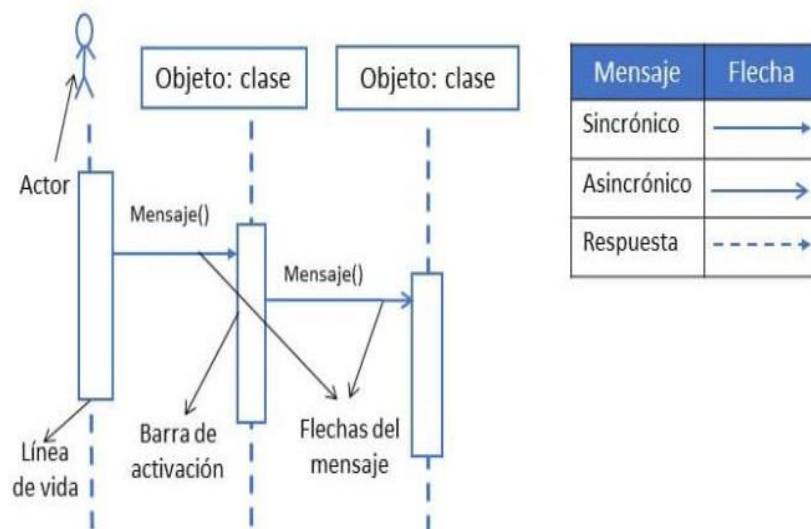


Figura 5: Diagrama de Secuencias

Fuente: Tomado de Ejemplos de Diagramas de Secuencias UML, WEByempresas(2022), <https://www.webyempresas.com/ejemplos-de-diagrama-de-secuencia/>

El **diagrama de secuencia** es un diagrama de interacción que muestra cómo actores, objetos o componentes intercambian mensajes a lo largo del tiempo. Los participantes se representan mediante líneas de vida y los mensajes se ordenan verticalmente desde los primeros hasta los últimos (Object Management Group, 2017).

Ventajas:

- Aclara el orden de las interacciones entre componentes.
- Permite identificar mensajes faltantes y dependencias innecesarias.
- Facilita relacionar un escenario con métodos y casos de prueba.

Desventajas:

- Puede ser difícil de leer cuando participan muchos objetos.
- Representa un escenario concreto y no toda la estructura del sistema.

- Necesita actualizarse cuando cambia el flujo de una funcionalidad.

2.3.2.4 Diagrama de Actividades

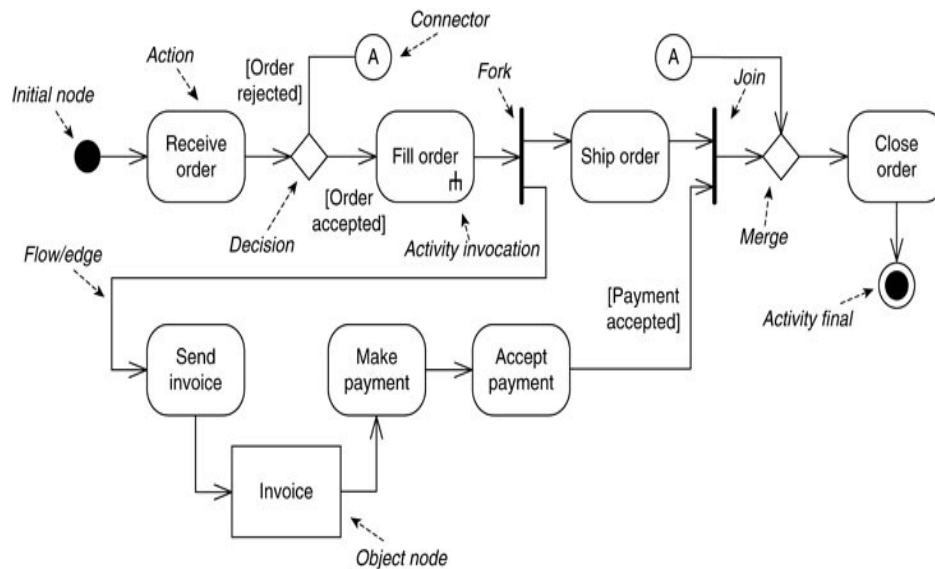


Figura 6: Diagrama de Actividades

Fuente: Tomado de, Cómo crear un diagrama de actividades, VENNGAGE(2024),
<https://es.venngage.com/blog/diagrama-de-actividades/>

El diagrama de actividades representa el flujo de acciones de un proceso, desde un nodo inicial hasta uno o varios nodos finales. Permite mostrar decisiones, condiciones, bifurcaciones, uniones, paralelismo y particiones de responsabilidad. Puede aplicarse a procesos de negocio o al comportamiento general de un caso de uso (Object Management Group, 2017).

Ventajas:

- Representa procesos lógicos, decisiones y rutas alternativas.
- Ayuda a detectar pasos innecesarios, repeticiones y puntos de bloqueo.
- Puede utilizarse durante el análisis y el diseño de la interfaz.

Desventajas:

- Un flujo extenso puede resultar difícil de interpretar.
- Puede confundirse con un diagrama de flujo si no se utiliza la notación adecuada.
- Las actividades paralelas requieren mayor cuidado en su representación.

2.3.3 Modelado de la Base de Datos NoSQL Documental

Una base de datos NoSQL documental almacena información en documentos formados por pares campo-valor, generalmente con una estructura semejante a JSON. Los documentos se agrupan en colecciones y pueden contener valores simples, listas, mapas y objetos anidados. Este modelo ofrece flexibilidad estructural y se adapta a datos que evolucionan durante el desarrollo (Sadalage y Fowler, 2013).

El modelado documental no debe describirse como “NoSQL relacional”, porque los modelos documental y relacional utilizan estructuras y criterios de diseño diferentes. En una base relacional, los datos se organizan principalmente en tablas y relaciones; en una base documental, la unidad principal es el documento y la información puede agruparse según la forma en que será consultada y modificada (Sadalage y Fowler, 2013).

Cloud Firestore es una base de datos NoSQL orientada a documentos. Organiza la información en colecciones y documentos; cada documento contiene campos y puede incluir mapas, arreglos o subcolecciones. Aunque no impone un esquema rígido, se recomienda mantener nombres y tipos de campos consistentes entre documentos de una misma colección para facilitar las consultas y el mantenimiento (Firebase, 2026b).

Elementos principales

- Colección: contenedor lógico que agrupa documentos de un mismo tipo o propósito.
- Documento: unidad de almacenamiento identificada por una clave única y compuesta por campos.
- Campo: propiedad que contiene un valor simple o complejo.
- Mapa o arreglo: estructura anidada utilizada para organizar valores relacionados dentro de un documento.
- Referencia: ruta o identificador que permite relacionar documentos sin utilizar uniones SQL.
- Subcolección: colección asociada a un documento para representar información jerárquica o potencialmente numerosa.
- Índice: estructura que permite ejecutar consultas y ordenamientos de manera eficiente.

Características

Las bases de datos documentales presentan características clave como:

- Esquema flexible, aunque requiere convenciones para conservar consistencia.

- Estructura jerárquica basada en colecciones, documentos y subcolecciones.
- Acceso a documentos por identificador y consultas sobre campos indexados.
- Capacidad de representar objetos anidados cercanos al modelo utilizado por la aplicación.
- Diseño condicionado por patrones de lectura, escritura y actualización.

Ventajas

- Facilita representar estructuras complejas dentro de un documento.
- Permite evolucionar campos sin realizar migraciones tradicionales de tablas.
- Reduce la necesidad de múltiples consultas cuando los datos relacionados se agrupan adecuadamente.
- Se adapta bien a aplicaciones móviles y servicios administrados en la nube.

Desventajas

- No dispone de uniones relacionales tradicionales.
- La duplicación de datos puede generar inconsistencias si no se controla.
- Las consultas posibles dependen de la estructura, los campos y los índices definidos.
- Un documento demasiado grande o una jerarquía excesiva puede dificultar lecturas y actualizaciones.
- El costo y el rendimiento pueden verse afectados por lecturas y escrituras innecesarias.

Diseño de colecciones y documentos

El diseño de colecciones y documentos define qué información se almacena junta, qué datos se separan y cómo se identifican las relaciones. La decisión debe considerar tamaño, frecuencia de actualización, crecimiento esperado, permisos y consultas. Los datos que se leen y modifican como una unidad pueden agruparse; los datos numerosos, independientes o con diferente ciclo de vida suelen separarse en otros documentos o subcolecciones (Firebase, 2026c).

Las colecciones deben tener una finalidad clara y nombres consistentes. Los documentos de una misma colección pueden ser flexibles, pero conviene mantener campos obligatorios, tipos de datos y convenciones comunes. También deben definirse identificadores, marcas de tiempo, estados y reglas para datos opcionales, evitando depender de campos inexistentes o con tipos diferentes (Firebase, 2026b).

2.3.3.1 Normalización y las primeras tres formas normales

Diseño orientado a consultas y desnormalización

El diseño orientado a consultas comienza identificando las preguntas que la aplicación debe responder y las operaciones que realizará con mayor frecuencia. A partir de ellas se definen colecciones, campos, índices y duplicaciones necesarias. Este criterio es importante en bases documentales porque no todas las consultas pueden resolverse mediante uniones posteriores, como ocurre en un sistema relacional (Sadalage y Fowler, 2013).

La desnormalización consiste en duplicar o agrupar intencionalmente determinados datos para reducir el número de lecturas y simplificar consultas. Por ejemplo, un documento histórico puede conservar una copia de los datos relevantes tal como estaban en el momento de la operación. Esta duplicación mejora la lectura y preserva contexto, pero obliga a definir cuál es la fuente principal y cómo se actualizarán las copias cuando corresponda (Sadalage y Fowler, 2013).

Ventajas del diseño orientado a consultas

- Reduce la cantidad de operaciones necesarias para mostrar información frecuente.
- Permite anticipar índices y límites antes de implementar las pantallas o servicios.
- Favorece estructuras adaptadas al uso real de la aplicación.

Desnormalización

La **desnormalización** es una técnica que consiste en combinar o duplicar intencionalmente algunos datos en una base de datos para mejorar el rendimiento de las consultas.

A diferencia de la normalización, que busca reducir la duplicación de datos, la desnormalización permite cierta repetición de información cuando esto ayuda a que las consultas sean más rápidas y requieran menos operaciones.

La desnormalización debe utilizarse de manera controlada, ya que puede provocar problemas de actualización y duplicación de información.

Riesgos de la desnormalización

- Aumenta el almacenamiento y el número de escrituras cuando un dato debe copiarse.
- Puede producir inconsistencias si no se identifica una fuente de verdad.
- Dificulta cambios masivos cuando una misma información está repetida en muchos documentos.
- Una duplicación sin justificación puede empeorar el mantenimiento y los costos.

2.4 Herramientas y Tecnologías de Implementación

Las tecnologías del proyecto se seleccionaron de acuerdo con el tipo de solución: una aplicación móvil para Android, con almacenamiento administrado en la nube, análisis facial en el dispositivo y un motor de recomendación explicable. Esta selección reduce la complejidad de infraestructura y mantiene coherencia entre los objetivos, el alcance y la implementación.

Cada herramienta cumple una función específica. Dart y Flutter conforman la aplicación; Firebase proporciona autenticación, datos y archivos; ML Kit y MediaPipe apoyan el análisis facial; Android Studio y Visual Studio Code conforman el entorno de desarrollo; GitHub conserva el historial; y Trello organiza el trabajo. La combinación debe evaluarse como un conjunto y no como tecnologías aisladas.

2.4.1 Lenguajes de Programación, IDE y Herramientas de Desarrollo

Lenguajes de Programación

2.4.1.1 Dart

Definición

Dart es un lenguaje de programación de código abierto desarrollado por Google y utilizado como lenguaje principal del framework Flutter. Es orientado a objetos, admite inferencia de tipos y utiliza comprobación estática para detectar incompatibilidades antes de la ejecución. También incorpora seguridad frente a valores nulos para reducir errores producidos por referencias no inicializadas (Dart, 2026).

El lenguaje incluye soporte para programación asíncrona mediante Future, Stream, async y await, lo que facilita trabajar con operaciones de red, archivos, bases de datos y eventos. Sus herramientas oficiales permiten analizar, formatear, probar y compilar código para diferentes plataformas (Dart, 2026).

Características

- Lenguaje orientado a objetos con clases, interfaces, herencia y composición.
- Sistema de tipos con seguridad frente a valores nulos.
- Programación asíncrona mediante Future, Stream y las palabras async y await.
- Compilación anticipada para producción y compilación en tiempo de ejecución durante el desarrollo.
- Herramientas oficiales de análisis estático, pruebas y formato de código.
- Integración directa con el framework Flutter.

Ventajas

- Permite compartir una misma base de código en las diferentes partes de una aplicación Flutter.
- El análisis estático ayuda a detectar errores antes de ejecutar la aplicación.
- La programación asíncrona facilita trabajar con Firebase, cámara y almacenamiento.
- El ecosistema incluye paquetes para arquitectura, pruebas y acceso a servicios móviles.

Desventajas

- Requiere aprendizaje específico para desarrolladores que no conocen Flutter.
- Algunas bibliotecas de visión por computadora pueden exigir integración nativa.
- La calidad y mantenimiento de paquetes de terceros no es uniforme.

2.4.2 Sistema Gestor de Base de Datos No Relacional (NoSQL)

Sistema Gestor de Base de Datos Documental (NoSQL) o Servicios Backend y Almacenamiento

2.4.2.1 Firebase Authentication

Firebase Authentication es un servicio de administración de identidad que permite autenticar usuarios mediante correo y contraseña, proveedores federados, teléfono y otros mecanismos. Sus SDK administran credenciales, sesiones y cambios en el estado de autenticación, y proporcionan un identificador único para cada usuario (Firebase, 2026a).

La autenticación comprueba la identidad, pero no define por sí sola qué operaciones puede realizar cada usuario. La autorización debe implementarse mediante roles, reglas de seguridad y validaciones. Firebase Authentication se integra con Cloud Firestore y Cloud Storage para utilizar la identidad dentro de sus reglas de acceso (Firebase, 2026a).

Características

- Administración de usuarios mediante identificadores únicos.
- Autenticación con correo electrónico y contraseña.
- Observación del estado de sesión en la aplicación.
- Recuperación y actualización de credenciales.
- Integración con Cloud Firestore, Cloud Storage y Security Rules.

Ventajas

- Evita almacenar contraseñas directamente en la base de datos de la aplicación.
- Reduce la cantidad de código necesario para implementar el inicio de sesión.
- Proporciona integración directa con los demás servicios de Firebase.

- Permite ampliar los métodos de autenticación en versiones futuras.

Desventajas

- La autenticación por sí sola no define qué funciones puede realizar cada usuario.
- Una configuración incorrecta de reglas puede exponer información.
- El servicio depende de la disponibilidad de la plataforma y de la conexión para ciertas operaciones.

2.4.2.2 Cloud Firestore

Cloud Firestore es una base de datos NoSQL documental administrada en la nube. Almacena documentos dentro de colecciones y permite realizar consultas, escuchar cambios en tiempo real, ejecutar transacciones y trabajar con caché local en plataformas compatibles (Firebase, 2026b).

Las consultas se apoyan en índices, por lo que la estructura de datos y las combinaciones de filtros deben planificarse. El acceso desde aplicaciones móviles y web se protege mediante Firebase Authentication y Cloud Firestore Security Rules, que pueden validar identidad, roles, campos y condiciones de lectura o escritura (Firebase, 2026b).

Características

- Modelo documental flexible y jerárquico.
- Consultas con filtros, ordenamiento, límites e índices.
- Listeners para recibir actualizaciones de datos.
- Caché local y sincronización en aplicaciones móviles.
- Operaciones por lotes y transacciones.
- Protección mediante Firebase Authentication y Security Rules.

Ventajas

- No requiere administrar un servidor de base de datos propio.
- Se integra directamente con Flutter mediante SDK oficiales.
- Facilita el crecimiento de colecciones y la sincronización de datos.
- Permite trabajar con estructuras cercanas a los objetos utilizados en Dart.

Desventajas

- No ofrece uniones SQL tradicionales.
- Los costos dependen del uso y deben supervisarse.
- El diseño de consultas e índices debe realizarse con anticipación.

- Una estructura deficiente puede producir lecturas innecesarias.

2.4.2.3 Firebase Storage

Cloud Storage for Firebase es un servicio de almacenamiento de objetos construido sobre Google Cloud Storage. Está orientado a guardar archivos como imágenes, videos, documentos y otros contenidos que no conviene incluir dentro de los documentos de una base de datos (Firebase, 2026d).

Los archivos se organizan mediante rutas y referencias. Los SDK permiten cargar, descargar, eliminar y consultar metadatos, mientras que las reglas de seguridad controlan quién puede acceder a cada ruta y bajo qué condiciones. La base de datos suele conservar únicamente la ruta, URL autorizada o metadatos necesarios para relacionar el archivo con otros registros (Firebase, 2026d).

Características

- Organización de archivos mediante rutas y referencias.
- Carga, descarga y eliminación desde aplicaciones móviles.
- Reanudación de transferencias ante problemas de conexión.
- Metadatos asociados a los objetos almacenados.
- Integración con Firebase Authentication y reglas de seguridad.

Ventajas

- Es apropiado para archivos que no deben guardarse dentro de documentos Firestore.
- Gestiona transferencias desde dispositivos móviles.
- Permite aplicar permisos por rutas y usuarios.
- Se integra con el ecosistema utilizado por la aplicación.

Desventajas

- El almacenamiento de imágenes aumenta el consumo y el costo del proyecto.
- Las rutas y reglas mal diseñadas pueden dificultar la administración.
- Se necesita una política para conservar o eliminar imágenes innecesarias.

2.4.3 Herramientas de Diseño

2.4.3.1 Android Studio

Android Studio es el entorno de desarrollo integrado oficial para aplicaciones Android. Incluye herramientas para administrar el SDK, crear emuladores, compilar proyectos,

inspeccionar registros, depurar código y analizar el rendimiento. También permite trabajar con proyectos Flutter mediante sus complementos (Android Developers, 2026).

Características

- Administración de versiones del SDK y herramientas de compilación.
- Emuladores para probar diferentes dispositivos y versiones de Android.
- Depuración, Logcat e inspección de recursos.
- Generación de archivos APK y Android App Bundle.
- Integración con Gradle, Git y GitHub.

Ventajas

- Proporciona una configuración completa para el desarrollo Android.
- Facilita diagnosticar problemas nativos y de compilación.
- Permite probar la aplicación en emuladores y dispositivos físicos.
- Ofrece herramientas oficiales para generar la versión instalable.

Desventajas

- Consume más recursos que un editor ligero.
- La configuración de SDK, Gradle y emuladores puede ser compleja.
- Las actualizaciones pueden exigir ajustes en dependencias del proyecto.

2.4.3.2 Visual Studio Code

Visual Studio Code es un editor de código fuente gratuito y extensible disponible para diferentes sistemas operativos. Incluye edición con IntelliSense, depuración, terminal integrada, control de versiones y un sistema de extensiones que añade compatibilidad con lenguajes, frameworks y herramientas (Microsoft, 2026).

Características

- IntelliSense y navegación por definiciones.
- Depuración con puntos de interrupción e inspección de variables.
- Terminal integrada para comandos de Flutter y Firebase.
- Integración con Git para revisar, confirmar y sincronizar cambios.
- Extensiones para lenguajes, formato, pruebas y administración del proyecto.

Ventajas

- Consume menos recursos que un IDE completo.
- Permite personalizar el entorno mediante extensiones.

- Facilita combinar edición de código, terminal y control de versiones.
- Es apropiado para trabajar diariamente con proyectos Flutter.

Desventajas

- Depende de extensiones para varias funciones especializadas.
- No reemplaza las herramientas completas del SDK de Android.
- Una configuración excesiva de extensiones puede afectar el rendimiento.

2.4.4 Frameworks y librerías

2.4.4.1 Flutter

Flutter es un framework de código abierto desarrollado por Google para construir aplicaciones compiladas desde una sola base de código. La interfaz se compone mediante widgets y puede adaptarse a distintas plataformas. En este proyecto se priorizará Android, debido al alcance y a los dispositivos disponibles para las pruebas (Flutter, 2026a).

Modelo MVVM en Flutter Características:

- **Modelo (Model):** Representa la lógica de acceso y manipulación de datos, generalmente asociado a una colección en la base de datos. Define la estructura de los datos y, en muchos casos, incluye métodos para convertir entre objetos Dart y documentos JSON (serialización/deserialización).
- **Vista (View):** Es la capa de presentación, compuesta por los Widgets de Flutter. Se encarga únicamente de mostrar la interfaz de usuario y capturar las interacciones, sin contener lógica de negocio. Se actualiza automáticamente al escuchar los cambios expuestos por el ViewModel.
- **VistaModelo (ViewModel):** Gestiona la lógica entre el modelo y la vista. Contiene el estado de la aplicación y la lógica de negocio, se comunica con el modelo para obtener o modificar datos, y expone esa información a la vista mediante herramientas de manejo de estado como Provider, Riverpod o ChangeNotifier.

Características

- Sistema declarativo de interfaces basado en widgets.
- Recarga rápida para visualizar cambios durante el desarrollo.
- Motor gráfico propio que ofrece una apariencia consistente.
- Soporte para pruebas unitarias, de widgets y de integración.
- Acceso a funciones nativas mediante plugins y canales de plataforma.
- Integración con herramientas de compilación para Android, iOS, web y escritorio.

Ventajas

- Permite desarrollar la aplicación móvil desde una sola base de código.
- Facilita construir interfaces consistentes y reutilizables.
- Dispone de integración amplia con Firebase y servicios de Android.
- Su arquitectura por widgets favorece la separación de pantallas y componentes.

Desventajas

- La aplicación puede tener un tamaño inicial mayor que una aplicación nativa sencilla.
- Algunas funciones dependen de paquetes externos y de su compatibilidad.
- Las integraciones especializadas pueden requerir código Kotlin o Java.

2.4.4.2 Google ML Kit Face Detection

ML Kit Face Detection es una API de Google para detectar rostros en imágenes o video. Puede obtener el rectángulo que contiene el rostro, ángulos de orientación y, según la configuración, puntos de referencia, contornos, clasificaciones básicas y seguimiento. La detección localiza rostros, pero no identifica a una persona ni realiza reconocimiento facial (Google Developers, 2026).

La API puede ejecutarse en el dispositivo y ofrece opciones que equilibran velocidad y precisión. La calidad de los resultados depende de resolución, iluminación, enfoque, tamaño visible del rostro y orientación. Los resultados deben validarse antes de utilizarlos en mediciones o decisiones posteriores (Google Developers, 2026).

Características

- Detección de uno o varios rostros en imágenes.
- Obtención del rectángulo facial y ángulos de orientación.
- Detección opcional de puntos y contornos.
- Procesamiento adecuado para aplicaciones móviles.
- Configuración de modos de rendimiento y precisión.
- Ejecución local en dispositivos compatibles.

Ventajas

- Permite validar rápidamente si la imagen contiene un rostro.
- Reduce transferencias de datos al ejecutar el procesamiento local.
- Dispone de documentación e integración con Android.

- Puede utilizarse como etapa previa a un análisis facial más detallado.

Desventajas

- La precisión depende de la iluminación, el enfoque, la distancia y la orientación.
- Los puntos disponibles pueden ser insuficientes para algunas mediciones detalladas.
- No determina por sí solo la forma facial ni la compatibilidad con una montura.

2.4.4.3 MediaPipe Face Landmarker

MediaPipe Face Landmarker es una solución de Google AI Edge que detecta puntos faciales detallados en imágenes, video o transmisión en vivo. Estos puntos describen regiones del rostro y permiten construir medidas relativas o efectos gráficos. MediaPipe fue concebido como un marco para desarrollar canalizaciones de percepción eficientes y multiplataforma (Google AI Edge, 2026; Lugaresi et al., 2019).

Características

- Obtención de una malla de puntos faciales detallados.
- Procesamiento de imágenes, video y flujo en vivo.
- Resultados que pueden utilizarse para calcular distancias y proporciones.
- Ejecución local en dispositivos compatibles.
- Modelos optimizados para tareas de visión en tiempo real.

Ventajas

- Ofrece mayor detalle geométrico que una detección facial básica.
- Permite calcular razones normalizadas entre diferentes zonas del rostro.
- Reduce la exposición de imágenes cuando el análisis se ejecuta localmente.
- Puede integrarse con un procedimiento de clasificación explicable.

Desventajas

- La integración con Flutter puede requerir paquetes o código nativo.
- Los resultados dependen de la calidad y posición de la imagen.
- Los puntos no proporcionan automáticamente una clasificación facial.
- Los umbrales deben ser definidos y validados con especialistas.

2.4.4.4 Motor de recomendación basado en reglas

Un sistema de recomendación busca seleccionar y ordenar alternativas relevantes a partir de información sobre el usuario, el contexto y los productos. En un enfoque basado en

reglas, el conocimiento se representa mediante condiciones y acciones que pueden excluir opciones o asignar puntuaciones. Esta estrategia favorece la explicación de por qué una alternativa fue sugerida (Ricci, Rokach y Shapira, 2022; Russell y Norvig, 2021).

Características

- Reglas expresadas mediante condiciones, acciones, pesos y prioridades.
- Posibilidad de excluir monturas incompatibles antes de calcular puntuaciones.
- Ordenamiento de resultados según la suma de criterios cumplidos.
- Registro de la versión de reglas utilizada en cada recomendación.
- Explicación resumida de los criterios principales aplicados.
- Actualización de reglas sin modificar toda la interfaz.

Ventajas

- El razonamiento puede ser revisado por especialistas.
- Las recomendaciones resultan más transparentes que una salida sin explicación.
- No requiere un conjunto grande de datos históricos para generar recomendaciones.
- Las reglas se administran y actualizan de acuerdo con los resultados de la validación profesional.

Desventajas

- La calidad depende directamente de las reglas y umbrales definidos.
- Un número elevado de reglas puede producir conflictos o mantenimiento complejo.
- No aprende automáticamente de nuevos casos si no se implementa un mecanismo adicional.
- Debe evitarse que una regla estética se presente como una obligación profesional.

2.4.5 Control de Versiones

2.4.5.1 Git y GitHub

Git es un sistema distribuido de control de versiones que registra cambios en archivos y permite trabajar con repositorios, commits, ramas y fusiones. Cada desarrollador conserva una copia local del historial, lo que facilita trabajar sin conexión, comparar versiones y recuperar estados anteriores (Chacon y Straub, 2014).

GitHub es una plataforma de alojamiento de repositorios Git que añade colaboración, revisión, incidencias, solicitudes de cambio, automatización y documentación. Git y GitHub no son lo mismo: Git administra versiones; GitHub ofrece servicios remotos alrededor de los repositorios Git (GitHub, 2026).

Características principales

- Repositorios locales y remotos.
- Commits que registran cambios identificables.
- Ramas para desarrollar funcionalidades sin afectar la versión estable.
- Comparación de cambios y resolución de conflictos.
- Issues y proyectos para seguimiento del trabajo.
- Etiquetas y versiones para marcar entregas importantes.

Ventajas

- Conserva evidencia del avance y las decisiones técnicas.
- Permite recuperar código cuando una modificación produce errores.
- Facilita trabajar por funcionalidades y mantener una rama estable.
- Proporciona respaldo remoto y documentación mediante README e Issues.

Desventajas

- Requiere disciplina para realizar commits claros y frecuentes.
- Los conflictos pueden ser difíciles para usuarios principiantes.
- Un archivo mal configurado puede exponer credenciales o datos sensibles.

2.4.6 Herramientas de Pruebas

2.4.6.1 Test Case

Un caso de prueba es una especificación de condiciones, datos, pasos y resultados esperados utilizada para verificar una función o atributo de calidad. Su finalidad es comprobar el comportamiento del sistema de manera repetible y dejar evidencia de los resultados obtenidos (Myers, Sandler y Badgett, 2011).

Características

- Identificador único y requerimiento asociado.
- Precondiciones necesarias antes de ejecutar la prueba.
- Datos de entrada y pasos ordenados.
- Resultado esperado definido antes de la ejecución.
- Resultado obtenido, estado y evidencia.
- Posibilidad de repetir el mismo escenario después de una corrección.

Ventajas

- Permite detectar defectos antes de la entrega final.

- Establece criterios claros para aceptar o rechazar una función.
- Facilita relacionar requerimientos, pruebas y evidencias.
- Ayuda a comprobar que una corrección no afectó funciones anteriores.

Desventajas

- Su elaboración y actualización requieren tiempo.
- Un caso mal definido puede producir resultados ambiguos.
- Los escenarios manuales deben complementarse con pruebas automatizadas cuando sea posible.

2.4.6.2 Casos de Prueba

Un caso de prueba es una especificación que define precondiciones, datos de entrada, pasos, resultado esperado y criterios para determinar si una función o atributo cumple lo requerido. Debe ser repetible y relacionarse con un requerimiento, historia de usuario, regla o riesgo específico (Myers et al., 2011).

Durante la ejecución se registra el resultado obtenido, el estado de la prueba, la evidencia y las incidencias encontradas. Los casos de prueba deben incluir escenarios correctos, valores límite, datos inválidos, errores esperados y condiciones de recuperación cuando corresponda (Myers et al., 2011).

Características

- Identificador único y referencia al requisito evaluado.
- Precondiciones y ambiente de ejecución.
- Datos de entrada y pasos ordenados.
- Resultado esperado definido antes de ejecutar.
- Resultado obtenido, estado y evidencia.

Ventajas

- Permite verificar funciones de manera repetible.
- Establece criterios claros para aceptar o rechazar un resultado.
- Facilita relacionar requerimientos, pruebas, defectos y correcciones.
- Ayuda a comprobar que un cambio no afectó comportamientos anteriores.

Desventajas

- Su elaboración y mantenimiento requieren tiempo.
- Un caso ambiguo produce resultados difíciles de interpretar.
- Los casos manuales repetitivos pueden ser costosos si no se automatizan.

2.4.6.3 Pruebas en Flutter

Flutter distingue pruebas unitarias, pruebas de widgets y pruebas de integración. Las pruebas unitarias verifican funciones o clases aisladas; las pruebas de widgets comprueban la construcción, interacción y estado de componentes visuales; y las pruebas de integración evalúan flujos completos en un dispositivo o entorno de aplicación (Flutter, 2026c).

Una estrategia equilibrada utiliza muchas pruebas unitarias y de widgets para obtener retroalimentación rápida, complementadas con pruebas de integración para los casos principales. Las pruebas automatizadas no sustituyen las evaluaciones de usabilidad ni las pruebas manuales exploratorias, pero ayudan a detectar regresiones y comportamientos inesperados (Flutter, 2026c).

Ventajas

- Permiten verificar lógica, interfaz y flujos en diferentes niveles.
- Detectan regresiones después de modificar código.
- Pueden ejecutarse de manera repetida y automatizada.

Limitaciones

- Requieren tiempo de diseño, programación y mantenimiento.
- Las pruebas de integración suelen ser más lentas y sensibles al entorno.
- Una cobertura alta no garantiza que los casos importantes estén correctamente evaluados.

2.4.7 Administración del Proyecto

2.4.7.1 Trello

Trello es una herramienta visual de administración de trabajo basada en tableros, listas y tarjetas. Su funcionamiento se relaciona con el enfoque Kanban y permite representar el estado de las tareas mediante columnas. Atlassian mantiene la plataforma y proporciona funciones para responsables, fechas, etiquetas y evidencias (Atlassian, 2026).

Características

- Tableros para separar proyectos o periodos de trabajo.

- Listas que representan estados del flujo.
- Tarjetas con descripción, responsable, fecha, etiquetas y archivos.
- Movimiento visual de tareas entre columnas.
- Historial de actividad y comentarios.
- Integraciones y automatizaciones según el plan utilizado.

Ventajas

- Ofrece una visión clara del trabajo pendiente, en proceso y terminado.
- Es sencillo de utilizar en un proyecto individual.
- Permite adjuntar capturas, criterios de aceptación y evidencias.
- Facilita representar el Product Backlog y el Sprint Backlog.

Desventajas

- La gestión de dependencias complejas es limitada.
- Algunas funciones avanzadas requieren planes de pago.
- No reemplaza la documentación técnica ni el control de versiones.